



BITS Pilani

Modern Database Systems

Prof. Ashish Narang
CSIS



BITS Pilani



<SSZG507, Modern Database Systems>

Lecture No. 2

- Introduction to SQL
- SQL in Relational Database Systems
- SQL Command Categories
- Data Definition Language (DDL) Statements
- Data Manipulation Language (DML) Statements
- Data Retrieval using the SELECT Statement
- SQL Joins and Subqueries
- Transaction Basics and ACID Properties
- Schema Evolution, Versioning, and Denormalisation
- Limitations of Relational Databases and Motivation for NoSQL

Why SQL?



- Relational databases store large volumes of structured data in tables. SQL provides a standard way to interact with this data efficiently and consistently.

SQL enables us to:

- **Create** databases and tables.
- **Insert** new records into tables.
- **Retrieve** specific information from one or more tables.
- **Update** existing records.
- **Delete** unwanted records.
- **Manage** transactions to maintain data integrity.
- **Control** user access and permissions.

Where SQL Fits in the Database Architecture

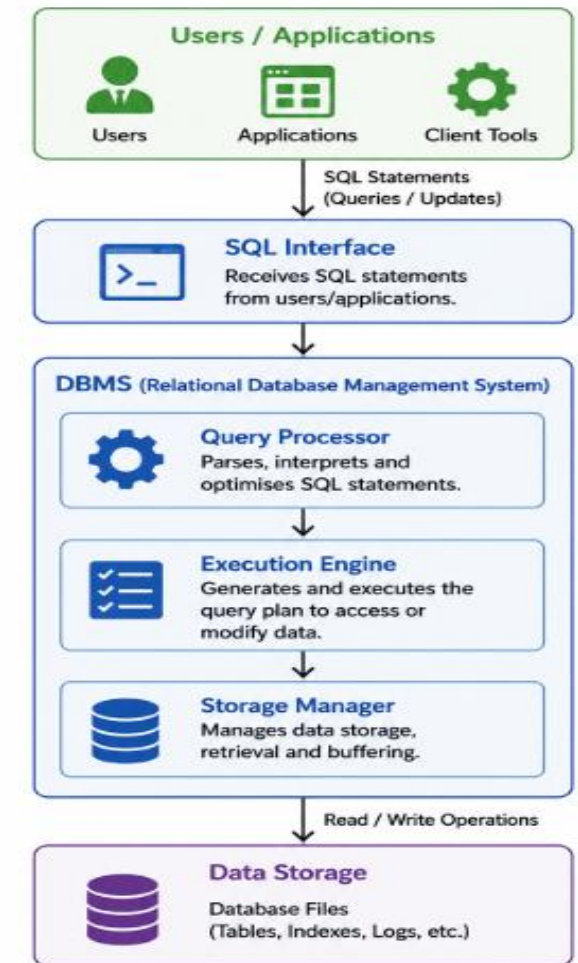


SQL in a Relational Database System

- SQL acts as the interface between users or applications and the relational database management system (RDBMS). Users issue SQL statements, which are processed by the DBMS to access or modify the data stored in the database.

Key Points

- Applications communicate with the database using SQL.
- The DBMS interprets and optimises SQL statements before execution.
- Data is physically stored by the storage manager.
- SQL provides a logical view of the data, hiding low-level storage details from users.



SQL Command Categories



- SQL provides different categories of commands, each serving a specific purpose in defining, manipulating, retrieving, securing, and managing data within a relational database.

Category	Purpose	Common Commands
DDL (Data Definition Language)	Defines and modifies the database schema and objects	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML (Data Manipulation Language)	Inserts, updates, and deletes data stored in tables	INSERT, UPDATE, DELETE, MERGE
DQL (Data Query Language)	Retrieves data from one or more tables	SELECT
TCL (Transaction Control Language)	Manages transactions to ensure reliable execution of database operations	COMMIT, ROLLBACK, SAVEPOINT
DCL (Data Control Language)	Controls user privileges and access to database objects	GRANT, REVOKE

Data Definition Language (DDL)



Data Definition Language (DDL) consists of SQL commands used to **define, modify, and manage the structure of database objects**. These commands affect the **database schema** rather than the data stored within the tables.

Command	Purpose
CREATE	Creates new database objects such as databases, tables, views, or indexes
ALTER	Modifies the structure of an existing database object
DROP	Permanently removes a database object
TRUNCATE	Removes all rows from a table while preserving its structure
RENAME	Changes the name of an existing database object (<i>supported in many DBMSs</i>)

CREATE TABLE Statement



- The **CREATE TABLE** statement is used to define a new table by specifying its columns, data types, and constraints.
- **General Syntax**

```
CREATE TABLE table_name (  
    column1 datatype [constraints],  
    column2 datatype [constraints],  
    ...  
    columnN datatype [constraints]  
);
```

- **Example**

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(100) NOT NULL,  
    Programme VARCHAR(50),  
    Semester INT,  
    CGPA DECIMAL(3,2)  
);
```

Element	Description
Table Name	Student
Columns	StudentID, StudentName, Programme, Semester, CGPA
Data Types	Define the type of data each column can store (e.g., INT, VARCHAR, DECIMAL)
Constraints	Enforce rules such as uniqueness, mandatory values, and primary keys

ALTER, DROP and TRUNCATE Statements



- Once a table has been created, SQL provides commands to modify its structure or remove it when it is no longer required.

- Alter Table Example

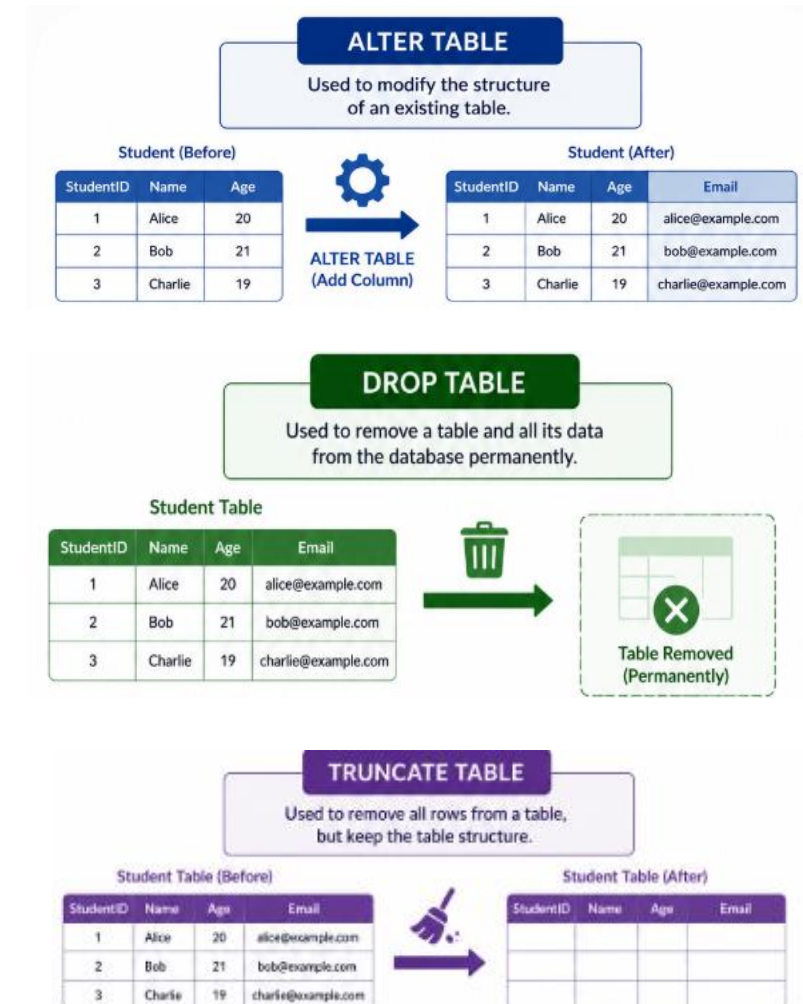
```
ALTER TABLE Student  
ADD Email VARCHAR(100);
```

- DROP TABLE

```
DROP TABLE Student;
```

- Truncate Table

```
TRUNCATE TABLE Student;
```



Data Manipulation Language (DML)



- **Data Manipulation Language (DML)** consists of SQL commands used to **insert, modify, delete, and merge data** stored in database tables. Unlike DDL, which defines the database structure, DML operates on the **records (rows)** within those tables.

Command	Purpose
INSERT	Adds new rows to a table
UPDATE	Modifies existing rows
DELETE	Removes one or more rows from a table
MERGE	Inserts or updates rows based on matching conditions (<i>supported in many DBMSs</i>)

INSERT Statement



- The **INSERT** statement is used to add one or more new rows to a table.

- Example

```
INSERT INTO table_name (column1, column2, ..., columnN)
VALUES (value1, value2, ..., valueN);
```

- Values must be provided in the same order as the specified columns, or the columns must be explicitly listed.

- Example

```
INSERT INTO Student
(StudentID, StudentName, Programme, Semester, CGPA)
VALUES (101, 'Rahul Sharma', 'M.Tech', 1, 8.75);
```

- Inserting Multiple Values

```
INSERT INTO Student
(StudentID, StudentName, Programme, Semester, CGPA)
VALUES
(102, 'Priya Gupta', 'M.Tech', 1, 8.90),
(103, 'Ankit Verma', 'M.Tech', 2, 8.45),
(104, 'Neha Singh', 'M.Tech', 1, 9.10);
```

Before Insertion					After Insertion				
StudentID	StudentName	Programme	Semester	CGPA	StudentID	StudentName	Programme	Semester	CGPA
100	Amit Kumar	M.Tech	1	8.20	100	Amit Kumar	M.Tech	1	8.20
101	Sneha Patel	M.Tech	1	8.50	101	Sneha Patel	M.Tech	1	8.50
					101	Rahul Sharma	M.Tech	1	8.75

Before Insertion					After Insertion				
StudentID	StudentName	Programme	Semester	CGPA	StudentID	StudentName	Programme	Semester	CGPA
100	Amit Kumar	M.Tech	1	8.20	100	Amit Kumar	M.Tech	1	8.20
101	Sneha Patel	M.Tech	1	8.50	101	Sneha Patel	M.Tech	1	8.50
					102	Priya Gupta	M.Tech	1	8.90
					103	Ankit Verma	M.Tech	2	8.45
					104	Neha Singh	M.Tech	1	9.10

UPDATE Statement



- The **UPDATE** statement modifies one or more existing rows in a table.

- General Syntax**

```
UPDATE table_name
SET column1 = value1,
    column2 = value2
WHERE condition;
```

- Example**

```
UPDATE Student
SET CGPA = 9.10
WHERE StudentID = 101;
```

Before Update

StudentID	StudentName	Programme	Semester	CGPA
100	Amit Kumar	M.Tech	1	8.20
101	Sneha Patel	M.Tech	1	8.50
102	Priya Gupta	M.Tech	2	8.45



Example:

```
UPDATE Student
SET CGPA = 9.10
WHERE StudentID = 101;
```

After Update

StudentID	StudentName	Programme	Semester	CGPA
100	Amit Kumar	M.Tech	1	8.20
101	Sneha Patel	M.Tech	1	9.10
102	Priya Gupta	M.Tech	2	8.45

Delete Statement



- The **DELETE** statement removes one or more rows from a table.

- General Syntax

```
DELETE FROM table_name  
WHERE condition;
```

- Example

```
DELETE FROM Student  
WHERE StudentID = 104;
```

Before Delete				
StudentID	StudentName	Programme	Semester	CGPA
100	Amit Kumar	M.Tech	1	8.20
101	Sneha Patel	M.Tech	1	9.10
102	Priya Gupta	M.Tech	1	8.90
104	Neha Singh	M.Tech	2	8.45

Example:

```
DELETE FROM Student  
WHERE StudentID = 104;
```

After Delete				
StudentID	StudentName	Programme	Semester	CGPA
100	Amit Kumar	M.Tech	1	8.20
101	Sneha Patel	M.Tech	1	9.10
102	Priya Gupta	M.Tech	1	8.90

Retrieving Data using the SELECT Statement



SELECT Statement

- The **SELECT** statement is used to retrieve data from one or more tables. It is the most frequently used SQL statement for querying information stored in a relational database.
- General Syntax

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Examples

1. Retrieve All Columns

```
SELECT *  
FROM Student;
```

2. Retrieve Specific Columns

```
SELECT StudentID, StudentName, CGPA  
FROM Student;
```

3. Retrieve Records Satisfying a specific condition

```
SELECT StudentName, Programme  
FROM Student  
WHERE Semester = 1;
```

Sample Table: Student

StudentID	StudentName	Programme	Semester	CGPA
100	Amit Kumar	M.Tech	1	8.20
101	Sneha Patel	M.Tech	1	9.10
102	Priya Gupta	M.Tech	1	8.90
103	Ankit Verma	M.Tech	2	8.45

1 Retrieve All Columns

```
SELECT *  
FROM Student;
```

Result

StudentID	StudentName	Programme	Semester	CGPA
100	Amit Kumar	M.Tech	1	8.20
101	Sneha Patel	M.Tech	1	9.10
102	Priya Gupta	M.Tech	1	8.90
103	Ankit Verma	M.Tech	2	8.45

2 Retrieve Specific Columns

```
SELECT StudentID, StudentName, CGPA  
FROM Student;
```

Result

StudentID	StudentName	CGPA
100	Amit Kumar	8.20
101	Sneha Patel	9.10
102	Priya Gupta	8.90
103	Ankit Verma	8.45

3 Retrieve Records Satisfying a Condition

```
SELECT StudentName, Programme  
FROM Student  
WHERE Semester = 1;
```

Result

StudentName	Programme
Amit Kumar	M.Tech
Sneha Patel	M.Tech
Priya Gupta	M.Tech

Common Clauses Used with SELECT



The SELECT statement can be combined with various clauses to filter, sort, group, and limit the data returned by a query.

Clause	Purpose	Example
FROM	Specifies the table(s) to retrieve data from	FROM Student
WHERE	Filters rows based on a condition	WHERE Semester = 1
DISTINCT	Eliminates duplicate values	SELECT DISTINCT Programme
ORDER BY	Sorts the result set	ORDER BY CGPA DESC
GROUP BY	Groups rows for aggregate calculations	GROUP BY Programme
HAVING	Filters groups after aggregation	HAVING AVG(CGPA) > 8.5
LIMIT / TOP	Restricts the number of rows returned	LIMIT 10

Examples



- Example 1:

```
SELECT StudentID, StudentName, Programme, CGPA
FROM Student
WHERE Semester = 1
      AND CGPA > 8.0
ORDER BY CGPA DESC;
```



Result			
StudentID	StudentName	Programme	CGPA
101	Sneha Patel	M.Tech	9.10
102	Priya Gupta	M.Tech	8.90
100	Amit Kumar	M.Tech	8.20

- Example 2:

```
SELECT Programme, AVG(CGPA) AS AverageCGPA
FROM Student
GROUP BY Programme
HAVING AVG(CGPA) > 8.5
ORDER BY AverageCGPA DESC;
```



Result	
Programme	AverageCGPA
M.Tech	8.97
B.Tech	8.58

Combining Data from Multiple Tables – Joins



SQL Joins

- A **JOIN** combines rows from two or more tables based on a related column. Joins allow data stored in separate tables to be retrieved as a single result set.

Student

<u>StudentID</u>	StudentName	Programme
------------------	-------------	-----------

Course

<u>CourseID</u>	CourseName	Credits
-----------------	------------	---------

Enrollment

<u>StudentID</u>	<u>CourseID</u>	Semester
------------------	-----------------	----------

Question: Which courses is each student enrolled in?

Types of SQL Joins



SQL provides different types of joins depending on how matching and non-matching rows from the participating tables should be returned.

Join Type	Description	Result
INNER JOIN	Returns only the rows that have matching values in both tables.	Matching rows only
LEFT OUTER JOIN	Returns all rows from the left table and the matching rows from the right table.	All left rows + matching right rows
RIGHT OUTER JOIN	Returns all rows from the right table and the matching rows from the left table.	All right rows + matching left rows
FULL OUTER JOIN	Returns all matching and non-matching rows from both tables.	All rows from both tables

INNER JOIN



- An **INNER JOIN** returns only those rows where the join condition is satisfied in **both** tables.

- Example

Student

StudentID	StudentName	Programme
101	Rahul Sharma	M.Tech
102	Priya Gupta	M.Tech
103	Ankit Verma	MBA

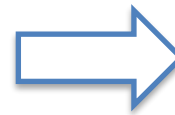
Enrollment

StudentID	CourseID
101	DB501
101	AI502
102	DB501

Result

StudentID	StudentName	CourseID
101	Rahul Sharma	DB501
101	Rahul Sharma	AI502
102	Priya Gupta	DB501

```
SELECT S.StudentID, S.StudentName,  
E.CourseID  
FROM Student S  
INNER JOIN Enrollment E  
ON S.StudentID = E.StudentID;
```



Comparison of SQL Join Types



- The choice of join determines whether matching rows, non-matching rows, or both are included in the query result.

Student

StudentID	StudentName	Programme
101	Rahul Sharma	M.Tech
102	Priya Gupta	M.Tech
103	Ankit Verma	MBA

Enrollment

StudentID	CourseID
101	DB501
101	AI502
102	DB501

Join Results

Join Type	Returned Rows
INNER JOIN	101, 102 (<i>matching rows only</i>)
LEFT OUTER JOIN	101, 102, 103 (<i>all students + matching enrollments</i>)
RIGHT OUTER JOIN	101, 102, 104 (<i>all enrollments + matching students</i>)
FULL OUTER JOIN	101, 102, 103, 104 (<i>all rows from both tables</i>)

- A **subquery** (also called a **nested query**) is a query written inside another SQL query. The result of the inner query is used by the outer query to perform filtering, comparison, or data retrieval.
- **Syntax**

```
SELECT column_name  
FROM table_name  
WHERE column_name operator  
(  
    SELECT column_name  
    FROM table_name  
    WHERE condition  
);
```

Common Uses of Subqueries

- Retrieve data based on the result of another query.
- Compare values with aggregated results (e.g., average, maximum, minimum).
- Filter records using IN, EXISTS, ANY, or ALL.
- Simplify complex retrieval problems.

Examples of Subqueries



Example 1: Single-Row Subquery

- Retrieve students whose CGPA is higher than the average CGPA of all students.

```
SELECT StudentID, StudentName, CGPA
FROM Student
WHERE CGPA >
(
    SELECT AVG(CGPA)
    FROM Student
);
```

Sample Tables		
Student		
StudentID	StudentName	CGPA
100	Amit Kumar	8.20
101	Sneha Patel	9.10
102	Priya Gupta	8.90
103	Ankit Verma	8.45
104	Neha Singh	7.80

Enrollment	
StudentID	CourseID
100	DB501
101	DB502
102	DB501
103	DB503
104	DB501
101	DB501

Result of Example 1

Average CGPA of all students = 8.49		
StudentID	StudentName	CGPA
101	Sneha Patel	9.10
102	Priya Gupta	8.90

Result of Example 2

Students enrolled in DB501	
StudentID	
100, 102, 104, 101	
StudentID	StudentName
100	Amit Kumar
101	Sneha Patel
102	Priya Gupta
104	Neha Singh

Example 2: Multi-Row Subquery

- Retrieve students enrolled in the course DB501.

```
SELECT StudentID, StudentName
FROM Student
WHERE StudentID IN
(
    SELECT StudentID
    FROM Enrollment
    WHERE CourseID = 'DB501'
);
```

Transaction Basics in Relational Systems



- A **transaction** is a sequence of one or more SQL operations that are executed as a **single logical unit of work**. A transaction is considered successful only if **all** its operations complete successfully; otherwise, all changes are undone.
- **Example: Bank Fund Transfer**
 - Transferring ₹5,000 from **Account A** to **Account B** involves two operations:

```
UPDATE Account
SET Balance = Balance - 5000
WHERE AccountID = 'A';

UPDATE Account
SET Balance = Balance + 5000
WHERE AccountID = 'B';
```

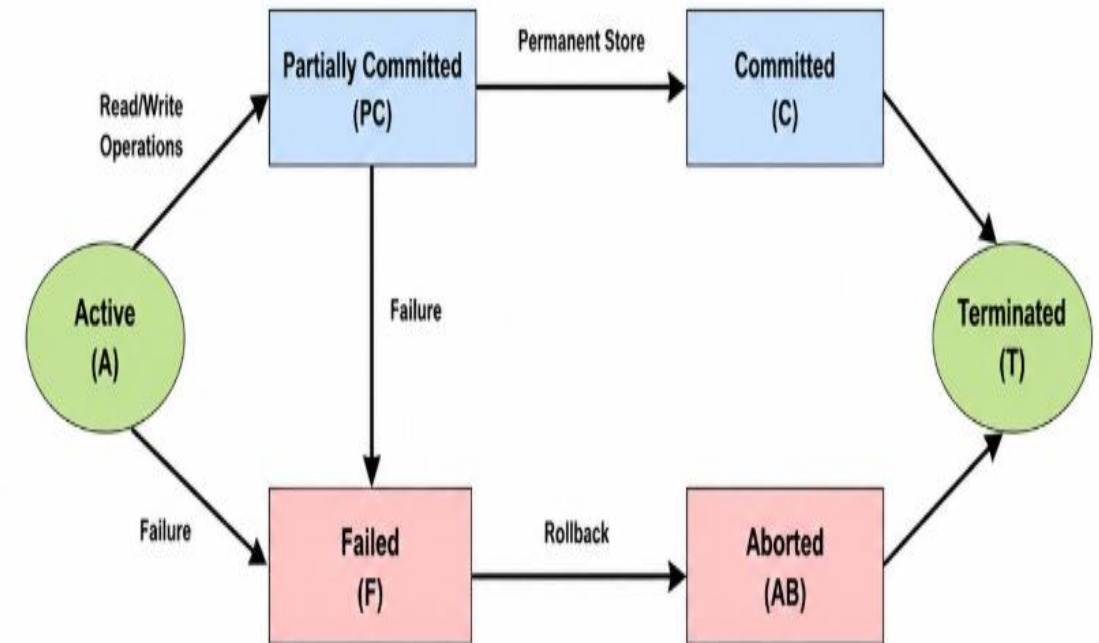
- **Key Characteristics**
 - A transaction may contain **one or more SQL statements**.
 - The database treats the entire transaction as **one unit of work**.
 - Either **all operations succeed**, or **none of them take effect**.

States of a Transaction



- During its execution, a transaction passes through a sequence of states before it either completes successfully or is terminated due to an error.

State	Description
Active	The transaction is currently executing one or more SQL statements.
Partially Committed	The final SQL statement has executed, but the changes have not yet been permanently recorded.
Committed	The transaction has completed successfully and all changes are permanently saved.
Failed	An error, system failure, or constraint violation prevents successful completion.
Aborted	All changes made by the transaction are rolled back, restoring the database to its previous consistent state.



Source: <https://medium.com/@matheesha27/transaction-states-acid-properties-in-a-dbms-94197273a14a>

ACID Properties of Transactions



- To ensure reliable and consistent transaction processing, relational database systems guarantee the **ACID** properties.

Property	Description	Example
Atomicity	A transaction is executed completely or not at all.	During a fund transfer, both the debit and credit operations succeed together; otherwise, both are rolled back.
Consistency	A transaction moves the database from one valid state to another while preserving integrity constraints.	The total amount of money in the banking system remains unchanged after a transfer.
Isolation	Concurrent transactions execute without interfering with each other.	Two users updating the same account should not produce incorrect balances.
Durability	Once a transaction is committed, its changes remain permanent even after a system failure.	A completed online payment is not lost after a server restart.

- **Schema evolution** refers to the process of modifying the structure of a database while preserving existing data and maintaining application compatibility.
- **Why is Schema Evolution Required?**
 - New business requirements introduce additional data attributes.
 - Existing attributes may need to be modified or removed.
 - Relationships between entities may change over time.
 - Performance and scalability requirements may require schema redesign.

Examples of Schema Changes

Change	SQL Operation
Add a new column	ALTER TABLE ... ADD COLUMN
Modify a column	ALTER TABLE ... MODIFY COLUMN
Remove a column	ALTER TABLE ... DROP COLUMN
Rename a table or column	ALTER TABLE ... RENAME

Schema Versioning

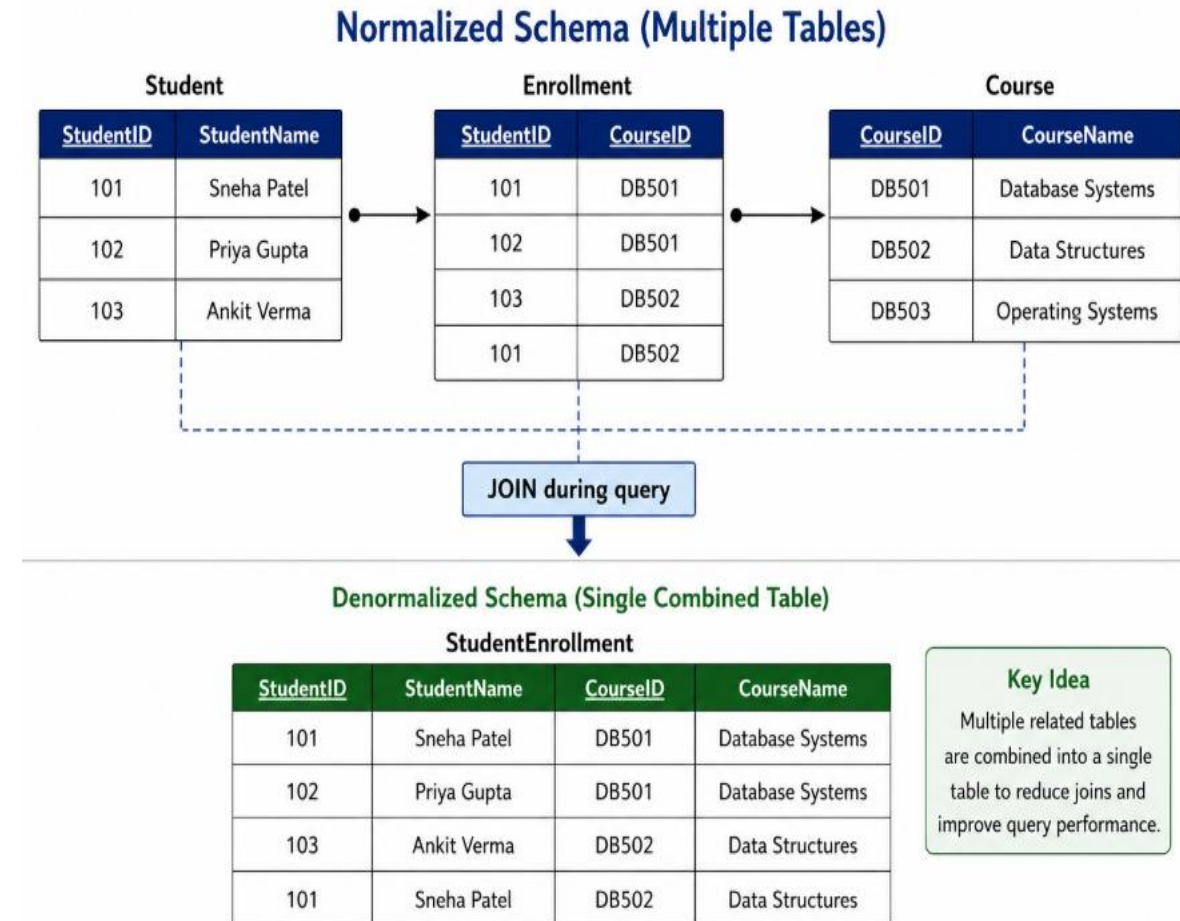


- **Schema versioning** is the practice of maintaining and tracking different versions of a database schema as it evolves. It enables databases and applications to evolve in a controlled manner while preserving compatibility and supporting reliable deployments.
- **Why is Schema Versioning Important?**
 - Tracks changes made to the database schema over time.
 - Ensures all environments (development, testing, and production) use the correct schema version.
 - Supports controlled deployment of database updates.
 - Enables rollback if a schema change introduces issues.
 - Facilitates collaboration among multiple developers.

Denormalisation



- **Denormalisation** is the process of **intentionally introducing redundancy** into a database schema to improve query performance and reduce the need for complex joins.
- **Why Denormalise?**
 - Reduce the number of table joins during query execution.
 - Improve read performance for frequently executed queries.
 - Simplify reporting and analytical queries.
 - Optimize performance for large-scale applications.



Limitations of Relational Database Systems



- Relational databases provide **strong consistency, structured data management, and powerful SQL capabilities**. However, modern applications often introduce requirements that are difficult to satisfy using traditional relational database systems alone.

Limitation	Example
Rigid Schema	Frequently changing data structures require schema modifications.
Horizontal Scalability	Scaling to millions of users often requires complex sharding strategies.
Semi-Structured Data	JSON, XML, logs, and multimedia data do not naturally fit relational tables.
High Write Throughput	IoT sensors, clickstreams, and social media generate massive volumes of writes.
Distributed Data Management	Maintaining ACID properties across geographically distributed systems can increase latency and complexity.



Thank You

